SINT Protocol: Runtime Authorization as a Safety Shield for LLM-Driven Physical Agents

Illia Pashkov

Independent Researcher · github.com/sint-ai/sint-protocol

1st IJCAI Workshop on Safe Physical AI (SPAI 2026) · Bremen, Germany · August 2026

Abstract. LLM-driven physical agents — systems where a foundation model issues commands to robots, drones, or industrial actuators — introduce a safety failure mode addressed by neither classical robot safety standards nor LLM alignment research: *an authenticated, aligned agent issuing a physically irreversible command without appropriate human oversight, in a degraded environment*. We present SINT Protocol, an open-source runtime authorization framework that interposes a cryptographically enforced safety shield at every agent–actuator boundary. SINT enforces six invariants: (1) capability confinement via signed Ed25519 tokens with physical constraint envelopes; (2) graduated human oversight across four approval tiers; (3) behavioral drift detection that makes the empirically observed 4.8× inter-model action-proposal variance [Cardenas et al., 2026] enforcement-visible; (4) environment-adaptive constraint tightening driven by real-time sensor state; (5) active detection plugins for prompt injection, memory poisoning, and supply-chain tampering; and (6) an EU AI Act–compliant emergency stop. Evaluated across 1,772 tests on 12 protocol bridges (ROS 2, MAVLink, MCP, OPC-UA, MQTT, A2A), the gateway adds <5 ms steady-state p99 latency — within the 10 ms budget of a 100 Hz ROS 2 control loop.

1. Introduction

The deployment of foundation models as controllers for physical systems is accelerating. Projects such as ROSClaw [1] demonstrate that GPT-4o, Claude 3.5, and Gemini 1.5 can drive ROS 2 robots through natural-language task specifications. However, the same study reveals a critical safety gap: across identical task envelopes, frontier LLMs exhibit up to **4.8× differences** in out-of-policy action proposal rates, with a 3.4× persistent divergence between the best and worst backends. This variance is *not* an alignment failure — the models are instruction-following correctly — but rather an intrinsic behavioral property that no amount of prompt engineering eliminates.

Classical robot safety standards (IEC 62443 [2], ISO 10218 [3]) assume a **trusted controller**: the software issuing commands is deterministic, formally verified, or at least version-locked. LLM-driven agents violate this assumption fundamentally. They are stochastic, prompt-injectable [4], and capable of behavioral drift across conversation turns. Meanwhile, LLM alignment research focuses on digital-only harms (toxic text, misinformation) and does not address the physical irreversibility of a `/cmd_vel` command already executed on hardware.

We identify three open problems at this intersection:

P1: The Authorization Gap. No existing framework enforces *who approved this physical action, under what constraints, with what evidence* — at runtime, cryptographically, before hardware execution.

P2: The Drift Visibility Gap. The 4.8× inter-model variance documented in [1] is currently invisible to the control system. An operator cannot distinguish a cautious agent from a reckless one until damage occurs.

P3: The Static Envelope Problem. Safety envelopes (max velocity, geofence) are necessarily static at configuration time, but the physical environment changes continuously. A fixed-parameter envelope is either

too restrictive or too permissive.

SINT Protocol addresses all three by interposing a runtime safety shield at the agent–actuator boundary. Every action flows through a single `PolicyGateway.intercept()` choke point that enforces capability tokens, approval tiers, behavioral drift detection, and environment-adaptive constraint tightening — before any command reaches hardware.

---

2. Architecture

2.1 Threat Model

We consider an LLM-driven agent that controls physical actuators via bridge adapters (ROS 2 topics, MAVLink commands, MCP tool calls, OPC-UA nodes, MQTT messages). The agent is *authenticated* (identity-verified) but *untrusted*: it may hallucinate dangerous commands, be prompt-injected, drift behaviorally, or have its tool chain tampered with. The human operator is trusted but may be absent, distracted, or overwhelmed by alert volume.

2.2 Safety Invariants

SINT enforces six safety invariants at every action boundary:

I-1: Capability Confinement. Every action requires presentation of a signed, expiring Ed25519 capability token scoped to: (a) a resource pattern (`ros2:///cmd_vel` , `mcp://bash`); (b) an action set (`publish` , `call`); and (c) a physical constraint envelope (max velocity in m/s, max force in N, geofence polygon, rate limit). Delegation is *attenuation-only*: child tokens can only narrow the parent's scope, never expand it. Token forgery requires breaking Ed25519 (computationally infeasible).

I-2: Graduated Human Oversight. Actions are classified into four tiers by physical consequence severity: T0 (observe, auto-allow), T1 (prepare, audited), T2 (act, requires human review), T3 (commit, requires explicit sign-off with optional M-of-N quorum). Classification uses 50+ pattern-matching rules. Shell-execution tools (`bash` , `exec` , `eval`) are hard-classified at T3 regardless of context, addressing OWASP ASI05 [4].

I-3: Behavioral Drift Detection (CSML). The Composite Safety-Model Latency score tracks: out-of-policy action rate (AR), boundary proximity score (BP), safety violation frequency (SV), and compliance rate (CR):

> $CSML(m,p,t) = \alpha \cdot AR_m + \beta \cdot BP_m + \gamma \cdot SV_m - \delta \cdot CR_m$

When CSML exceeds threshold θ, all subsequent requests from that agent are automatically escalated by one tier — making the 4.8× inter-model variance [1] a *live enforcement trigger* rather than a post-incident finding.

I-4: Environment-Adaptive Constraint Tightening. The `DynamicEnvelopePlugin` maps real-time sensor state to tighter effective limits: v_eff = min(token.maxVelocity, d_obs · r_f), where d_obs is the nearest obstacle distance and r_f is a reaction factor. This closes the gap identified in [1] between a "configurable" and an "environment-adaptive" safety envelope.

I-5: Active Threat Detection Plugins. The gateway hosts pluggable detectors: *GoalHijackPlugin* (ASI01: 5-layer prompt injection detection), *MemoryIntegrityChecker* (ASI06: replay attacks, cross-session continuity injection, credential read funnels, action velocity loops), *ArgInjectionDetector* (ASI05: shell metacharacters, path traversal), and *SupplyChainVerifier* (ASI04: model fingerprint hash validation).

I-6: Emergency Stop (EU AI Act Art. 14(4)(e)). The `CircuitBreakerPlugin` implements a three-state circuit (CLOSED → OPEN → HALF_OPEN): manual operator trip, automatic trip on N consecutive denials or

anomalous CSML, and probe-based recovery.

2.3 Swarm Collective Safety

For multi-agent deployments, a `SwarmCoordinator` enforces collective constraints *before* individual gateway calls: maximum concurrent T2+ actors, total kinetic energy ceiling ($\Sigma \frac{1}{2} m_i \cdot v_i^2 \leq E_{max}$), minimum inter-agent distance, and maximum escalated fraction. These constraints are fundamentally collective — no per-agent token can express "at most 3 of 10 agents may be in T2 simultaneously."

---

3. Evaluation

3.1 Latency

The gateway must not violate the timing budget of real-time control loops. We benchmark on a ROS 2 `/cmd_vel` path at 100 Hz (10 ms budget) on Apple M3 Pro:

Metric	p50	p95	p99	Steady p99
Latency (ms)	1.5	5.3	12.5	**5.1**

The steady-state p99 of 5.1 ms consumes 51% of the 10 ms budget, leaving headroom for bridge serialization and network transport. Cold-start p99 reaches 20 ms due to JIT; amortized after ~120 iterations.

3.2 Test Coverage and Conformance

Category	Tests
Policy gateway (tiers, constraints, plugins)	256
OWASP ASI01–ASI10 conformance fixtures	29
Bridge adapters (12 protocols)	233
Capability tokens + delegation	58
Evidence ledger (hash chain integrity)	59
Swarm, economy, persistence, engine	951
Integration + end-to-end	186
Total	**1,772**

The OWASP conformance suite contains 29 machine-readable fixture pairs (attack vector + safe-case control) covering all 10 ASI categories [4]. Each fixture specifies a `SintRequest`, token scope, and expected gateway decision.

3.3 Bridge Protocol Coverage

Physical: ROS 2, MAVLink v2, MQTT/CoAP, OPC-UA, Open-RMF, Sparkplug B. **Agent:** MCP, Google A2A, gRPC.

---

4. Self-Critique and Open Problems

L-1: No formal verification. SINT's invariants are enforced by runtime code, not formal proof. A TLA+ mechanization of the 12-state DFA would prove that physical actuation is unreachable without a valid token (invariant I-G1: No Bypass). Planned for Q3 2026.

L-2: No real-robot evaluation. All benchmarks run in TypeScript simulation. The 5.1 ms p99 has not been validated on a physical ROS 2 system with network transport and actuator feedback. A warehouse AMR pilot (Clearpath Jackal + UR5e) is planned.

L-3: CSML threshold empirically ungrounded. The escalation threshold $\theta = 0.3$ is a design parameter, not empirically validated. Optimal thresholds require deployment data from diverse model backends under varied task distributions.

L-4: Heuristic threat detection. GoalHijackPlugin and MemoryIntegrityChecker use pattern-matching heuristics, not learned classifiers. Sophisticated adversarial prompts may evade detection. A hybrid heuristic + lightweight transformer approach is a natural extension.

L-5: Static tier classification. The 50+ tier rules are hand-authored. A learned classifier trained on operator approval decisions could reduce false escalations and adapt to deployment-specific risk profiles.

---

5. Related Work

Robot safety standards. IEC 62443 [2] and ISO 10218 [3] define security requirements for industrial systems but assume trusted controllers. SINT extends these to the untrusted LLM-agent case.

Safety shields. Alshiekh et al. [6] formalize runtime monitors that override unsafe RL actions. SINT shares the interposition architecture but operates at capability-token/approval-tier abstraction, supporting multi-protocol multi-agent deployments.

Agentic AI security. OWASP ASI [4] identifies 10 risk categories. Microsoft's Agent Governance Toolkit (2026) addresses these for digital agents. SINT is the first framework providing verified ASI01–ASI10 coverage for *physical* agents.

Capability-based security. The object-capability model [7] provides SINT's token design foundation. Unlike RBAC, capability tokens are unforgeable, attenuatable, and revocable.

LLM-driven robotics. ROSClaw [1] provides SINT's empirical foundation. SayCan [8] and Code-as-Policies [9] demonstrate LLM-to-robot pipelines with no runtime authorization.

---

6. Conclusion

We presented SINT Protocol, a runtime safety shield for LLM-driven physical agents that enforces capability confinement, graduated human oversight, behavioral drift detection, environment-adaptive constraints, active threat detection, and emergency stop — at sub-5 ms steady-state p99 latency on a 100 Hz ROS 2 control loop.

Core position: Safe physical AI requires authorization as a *first-class runtime concern*, enforced cryptographically at the agent–actuator boundary, not as deployment configuration or prompt engineering.

SINT is open source (Apache 2.0): github.com/sint-ai/sint-protocol

---

References

[1] I. S. Cardenas et al., "ROSClaw: An OpenClaw ROS 2 Framework for Agentic Robot Control and Interaction," *arXiv:2603.26997*, 2026.

[2] IEC 62443-3-3:2013, "Industrial communication networks — IT security for networks and systems."

[3] ISO 10218-1:2011, "Robots and robotic devices — Safety requirements for industrial robots."

[4] OWASP Foundation, "OWASP Top 10 for Agentic Applications," 2025.

[5] NIST, "AI Risk Management Framework (AI RMF 1.0)," 2023.

[6] M. Alshiekh et al., "Safe Reinforcement Learning via Shielding," *Proc. AAAI*, 2018.

[7] M. S. Miller et al., "Capability Myths Demolished," Tech. Rep., 2003.

[8] M. Ahn et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," *Proc. CoRL*, 2022.

[9] J. Liang et al., "Code as Policies: Language Model Programs for Embodied Control," *Proc. ICRA*, 2023.

[10] NIST SP 800-82 Rev. 3, "Guide to Operational Technology (OT) Security," 2023.